

Conceitos Básicos

Introduz conceitos necessários ao entendimento das diferentes estruturas a serem vistas ao longo deste curso

Conceitos Básicos

**Tipos de dados e
Estruturas de dados**

Tipos de Dados

Tipo de dado

definição do conjunto de valores (domínio)
que uma variável pode assumir

Ex: inteiro

< ... -2, -1, 0, +1, +2, ... >

lógico

< verdadeiro, falso >

Tipos de Dados

- **Tipos básicos (primitivos)**
 - inteiro, real, e caractere
- **Tipos de estruturados (construídos)**
 - arranjos (vetores e matrizes)
 - estruturas
 - seqüências (conjuntos)
 - referências (ponteiros)
- **Tipos definidos pelo usuário**

Tipos e Estruturas de Dados

- **Tipos de dados básicos**
 - Fornecidos pela Linguagem de Programação
- **Estruturas de Dados**
 - Estruturação conceitual dos dados
 - Reflete um **relacionamento lógico** entre dados, de acordo com o problema considerado

Exemplo de estrutura de dados: Lista linear

- Relação de ordem entre os dados
- Linear - seqüencial



Ex:

aplicação: empresa

problema: dados dos funcionários – cada nó um funcionário

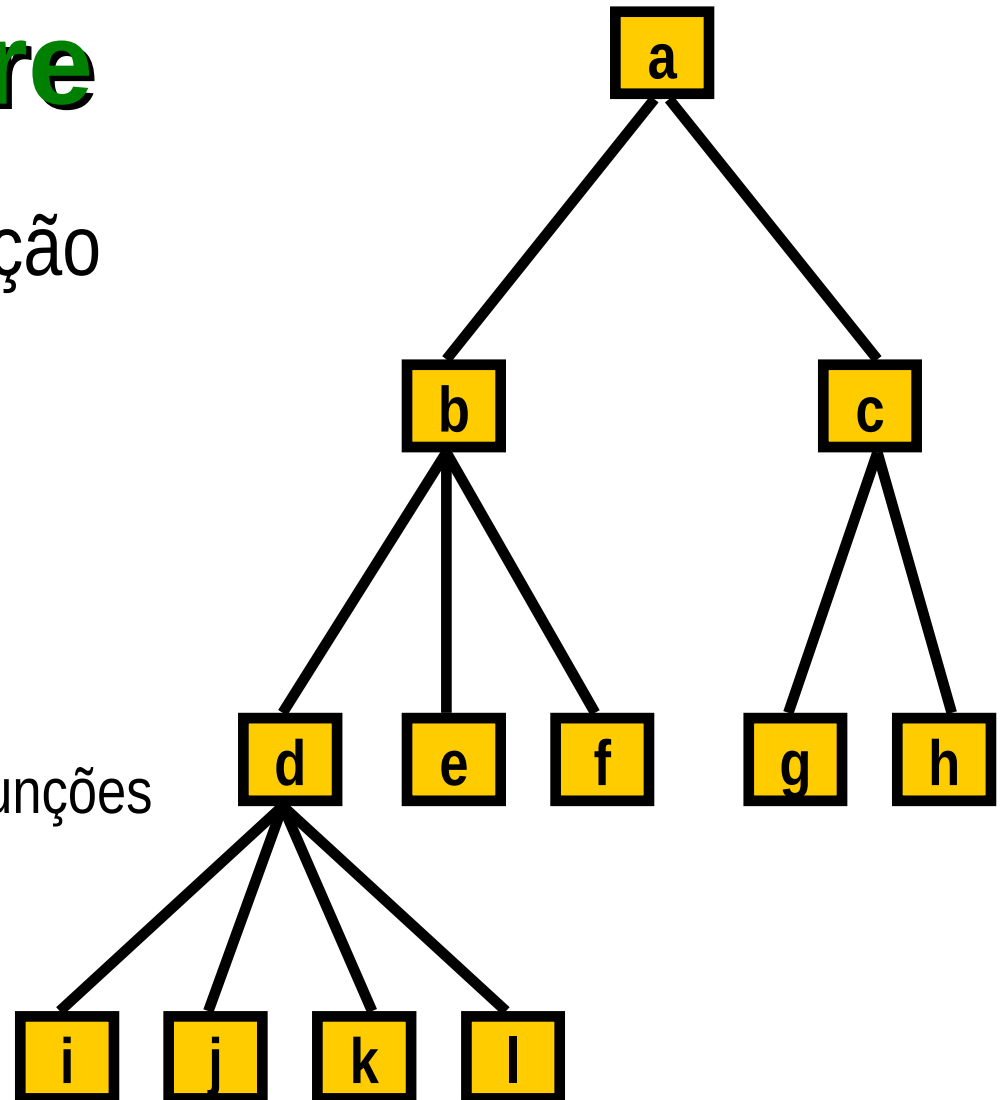
Exemplo de estrutura de dados : **Árvore**

- Relação de subordinação entre os dados

Ex:

aplicação: empresa

problema: organograma de funções



Operações sobre estruturas de dados

Estruturas de Dados incluem as **operações** para a manipulação de seus dados

Operações básicas:

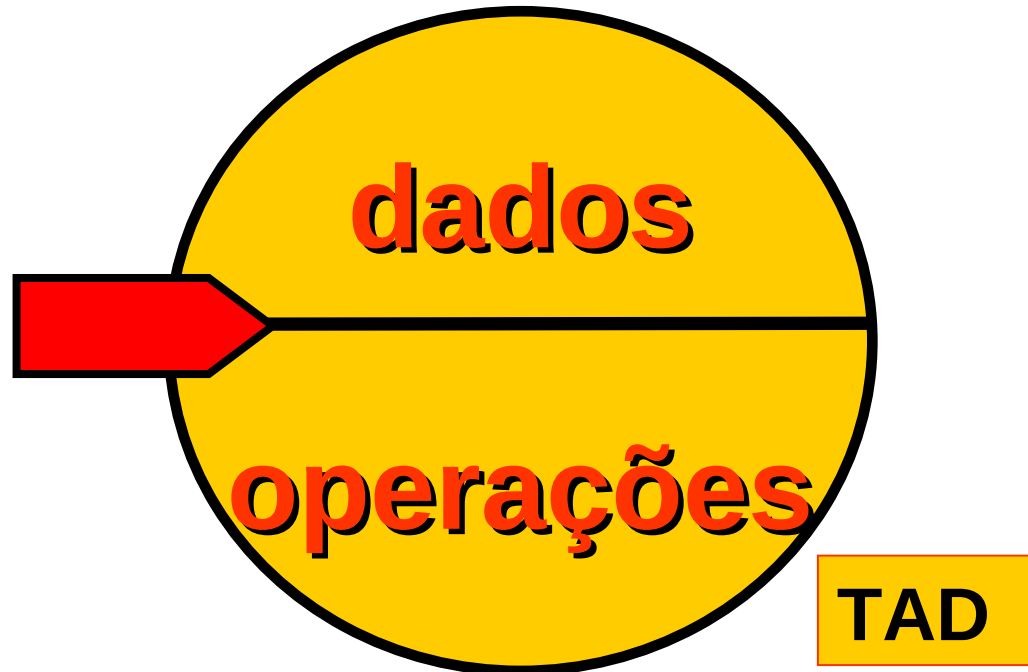
- criação da estrutura de dados
- inclusão de um novo elemento
- remoção de um elemento
- acesso a um elemento
- destruição da estrutura de dados

Conceitos Básicos

TADs - Tipos

Abstratos de Dados

Tipos Abstratos de Dados



TADs

Um **TAD** é uma forma de definir um **novo tipo** de dado juntamente com as **operações** que manipulam esse novo tipo de dado

TADs

- Separação entre conceito (definição do tipo) e implementação das operações
- Visibilidade da estrutura interna do tipo fica limitada às operações
- Aplicações que usam o TAD são denominadas *clientes* do tipo de dado
- Cliente tem acesso somente à forma abstrata do TAD

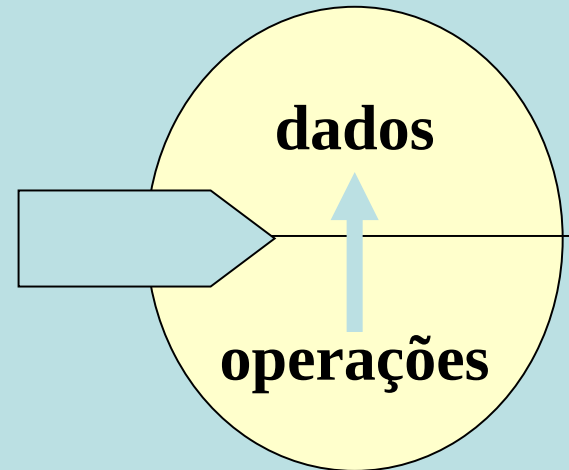
TADs

Um **TAD** (em LP) é um tipo de dado que satisfaz as condições:

- A representação ou a definição do tipo e as operações sobre variáveis desse tipo **estão contidas numa única unidade sintática**
 - **MÓDULO**
- A representação interna do tipo (a implementação) **não é visível de outras unidades** sintáticas, de modo que só as operações oferecidas na definição do tipo podem ser usadas com as variáveis desse tipo

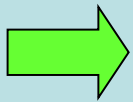
Propriedades dos TADs

- Satisfazem as propriedades de
 - **encapsulamento**: definição isolada de outras unidades do programa
 - **invisibilidade** e **proteção**: representação do tipo deve ser acessada somente no ambiente encapsulado
- A LP deve possibilitar
 - ambiente encapsulado
 - proteção de dados
 - interface para acesso
 - operações básicas



Vantagens de TADs

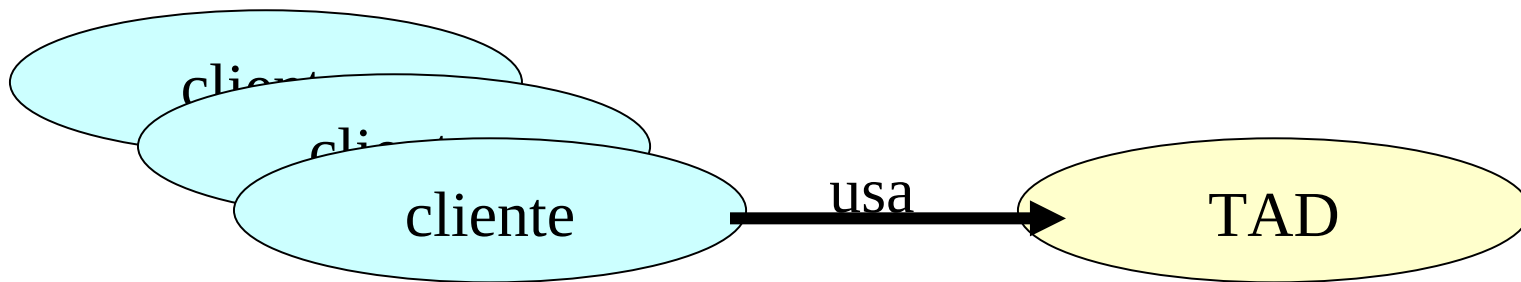
- Possibilidade de utilização do mesmo TAD em diversas aplicações diferentes
- Possibilidade de alterar o TAD sem alterar as aplicações que o utilizam



REUTILIZAÇÃO

Vantagens de TADs

- Código do cliente do TAD não depende da implementação
- **Segurança:**
 - clientes não podem alterar a representação
 - clientes não podem tornar os dados inconsistentes

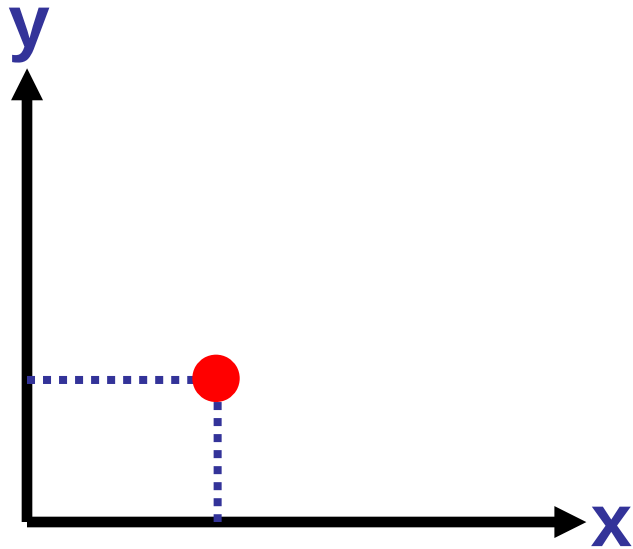


Projeto de um TAD

Envolve a escolha de operações adequadas para uma determinada estrutura de dados, definindo seu comportamento

- **Dicas para definir um TAD:**
 - definir pequeno número de operações
 - conjunto de operações deve ser suficiente para realizar as computações necessárias às aplicações que utilizarem o TAD
 - cada operação deve ter um propósito bem definido, com comportamento constante e coerente

Exemplo de TAD: representação de um ponto



- **Modelo**
Par ordenado (x,y)
- **Dados**
representando o modelo
 - Coordenada X
 - Coordenada Y

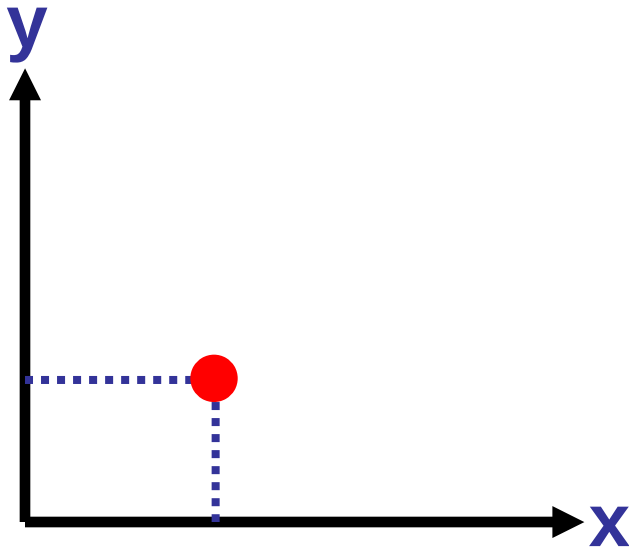
Exemplo de TAD: ponto

- Par (v, o)
 - v – dupla formada por dois reais
 - o – operações aplicáveis sobre o tipo Ponto

Exemplo de TAD: representação de um ponto

Operações:

- **pto_cria**: operação que cria um ponto, alocando memória para as coordenadas x e y;
- **pto_libera**: operação que libera a memória alocada por um ponto;
- **pto_acessa**: operação que devolve as coordenadas de um ponto;
- **pto_atribui**: operação que atribui novos valores às coordenadas de um ponto;
- **pto_distancia**: operação que calcula a distância entre dois pontos.



Exemplo de TAD: representação de um ponto

TAD Ponto

Dado: consiste de um par ordenado de
numeros reais (x,y)

Operações:

pto_cria

entrada: a abscissa e a ordenada do ponto sendo criado

processo: aloca dinamicamente a estrutura que representa um ponto e inicializa seus campos de acordo com os parametros de entrada

saída: retorna o endereço da estrutura alocada

pto_libera

entrada: o endereço do ponto

processo: libera a area de memoria
alocada para o ponto cujo endereço foi
passado como parametro

saida: o endereço do ponto

pto_acessa

entrada: o endereço do ponto e o endereço das variáveis reais x e y

processo: atribui às variáveis x e y os valores da abscissa e ordenada, respectivamente, do ponto cujo endereço foi passado como parametro

saida: retorna 1 se sucesso e 0 se fracasso (qdo o ponto nao existe).

pto_atribui

entrada: o endereço do ponto e as variáveis reais x e y

processo: atribui à abscissa do ponto, cujo endereço foi passado como parametro, o valor de x e à sua ordenada, o valor de y

saida: retorna 1 se sucesso e 0 se fracasso (qdo o ponto nao existe).

pto_distancia

entrada: o endereço de dois pontos

processo: calcula a distancia entre os pontos cujos endereços foram passados como parametro

saida: retorna a distancia calculada ou -1 se fracasso (um ou os dois pontos nao existem)

Implementação de um TAD

- Deve ser implementado independente do programa de aplicação (**cliente**);
- Todas as funções que implementam as operações definidas para um TAD devem ser definidas num arquivo com extensão **.c**
- Os protótipos das funções devem ser declarados em um arquivo com o mesmo nome do arquivo a que está associado, com extensão **.h** (*Interface do modulo com extensão .c*)

Implementação de um TAD

A interface ponto.h

- inclui os protótipos das funções que implementam as operações definidas no **TAD Ponto**;
- define o nome do tipo de dados



- os módulos que utilizarem o TAD Ponto:
 - >> não poderão acessar diretamente os campos da estrutura Ponto
 - >> só terão acesso aos dados obtidos através das funções exportadas

Interface ponto.h

```
/* TAD: Ponto (x,y) */  
/* Tipo exportado */  
typedef struct ponto Ponto;  
  
/* Funções exportadas */  
/* Função cria - Aloca e retorna um ponto com coordenadas (x,y) */  
Ponto* pto_cria (float x, float y);  
  
/* Função libera - Libera a memória de um ponto previamente criado */  
void pto_libera (Ponto* p);  
  
/* Função acessa - Retorna os valores das coordenadas de um ponto */  
void pto_acessa (Ponto* p, float* x, float* y);  
  
/* Função atribui - Atribui novos valores às coordenadas de um ponto */  
void pto_atribui (Ponto* p, float x, float y);  
  
/* Função distancia - Retorna a distância entre dois pontos */  
float pto_distancia (Ponto* p1, Ponto* p2);
```

[ponto.h](#) - arquivo com a interface de *Ponto*

Implementação de ponto.c

- inclui o arquivo de interface de Ponto (ponto.h)
- define a composição da estrutura Ponto
- inclui a implementação das operações da TAD Ponto.

Implementação de ponto.c

```
#include <stdlib.h>
#include <math.h>
#include "ponto.h"
```

```
struct ponto {
    float x;
    float y;
};
```

```
Ponto* pto_cria (float x, float y)
{ }
void pto_libera (Ponto* p)
{ }
void pto_acessa (Ponto* p, float* x, float* y)
{ }
void pto_atribui (Ponto* p, float x, float y)
{ }
float pto_distancia (Ponto* p1, Ponto* p2)
{ }
```

Implementação de ponto.c

- Função para criar um ponto dinamicamente:
 - aloca a estrutura que representa o ponto
 - inicializa os seus campos

```
Ponto* pto_cria (float x, float y)
{
    Ponto* p = (Ponto*) malloc(sizeof(Ponto));
    if (p != NULL)
    {
        p->x = x;
        p->y = y;
    }

    return p;
}
```


Implementação de ponto.c

- Função para liberar um ponto:
 - deve apenas liberar a estrutura criada dinamicamente através da função *cria*

```
void pto_libera (Ponto* p)
{
    free(p);
}
```

Implementação de ponto.c

- Funções para acessar e atribuir valores às coordenadas de um ponto:
 - permitem que uma função cliente acesse as coordenadas do ponto sem conhecer como os valores são armazenados

```
void pto_acessa (Ponto* p, float* x, float* y)
{
    *x = p->x;
    *y = p->y;
}
```

```
void pto_atribui (Ponto* p, float x, float y)
{
    p->x = x;
    p->y = y;
}
```

Implementação de ponto.c

- Função para calcular a distância entre dois pontos

```
float pto_distancia (Ponto* p1, Ponto* p2)
{
    float dx = p2->x - p1->x;
    float dy = p2->y - p1->y;
    return sqrt(dx*dx + dy*dy);
}
```

Exemplo de um cliente

Implemente um programa, que dados dois pontos pela linha de comando, imprima a distancia entre eles.

Use a TAD Ponto na implementação.

Exemplo de um cliente

```
#include <stdio.h>
#include <stdlib.h>
#include "ponto.h"
int main(int argc, char **argv)
{
    if (argc != 5)
    {
        printf("\n entre com as coordernadas dos pontos...");
        getchar();
        exit(1);
    }
    float d;
    Ponto *p,*q;
    p = pto_cria(atof(argv[1]),atof(argv[2]));
    q = pto_cria(atof(argv[3]),atof(argv[4]));
    d = pto_distancia(p,q);
    printf("Distancia entre pontos: %5.2f\n",d);
    pto_libera(q);
    pto_libera(p);
    getchar();
    return 0;
}
```

Implementação do TAD Ponto

Usando o gcc, os módulos são compilados e linkados da seguinte forma:

```
gcc -c ponto.c
```

```
gcc -c prog.c
```

```
gcc -o project2 prog.o ponto.o
```

Exercício

Definir o TAD N_Racionais:

TAD N_Racionais

Dado:

Operações:

OP1:

 entrada:

 processo:

 saída:

.....

Exemplo de TAD: N_Racionais

- `cria_rac`

entrada: nenhuma

processo: aloca área de memória para armazenar um numero racional

saída: o endereço da área reservada em caso de sucesso ou uma condição inválida em caso de fracasso.

Exemplo de TAD: N_Racionais

- libera_rac

entrada: endereço de um numero racional

processo: libera área associada ao endereço passado como parâmetro

saida: retorna uma condição inválida indicando area liberada

Exemplo de TAD: N_Racionais

- atribui_rac

entradas: endereço de um numero racional , os valores para numerador e o denominador

processo: atribui o novo valor ao numero racional cujo endereço foi passado como parâmetro

saida: retorna 1 se sucesso, 0 se fracasso (se denominador == 0 ou se o endereço do numero racional passado como parâmetro for inválido)

Exemplo de TAD: N_Racionais

- `acessa_rac`

entradas: o endereço de numero racional, endereço do numerador e endereço do denominador

processo: atribui aos parâmetros os valores do numerador e denominador do numero racional cujo endereço foi passado como parâmetro

saída: 1 se sucesso e 0 se fracasso (numero racional invalido)

Exemplo de TAD: N_Racionais

- soma_rac

entrada: o endereço de dois numeros racionais n1, n2 e o endereço do número racional resultante nr

processo: denominador de nr = denominador de n1 * denominador de n2;

o numerador de nr = numerador de n1*denominador de n2 + numerador de n2* denominador de n1

saida: 1 se sucesso e 0 se fracasso (se n1, ou n2 ou nr forem invalidos)

Exemplo de TAD: N_Racionais

- subtrai_rac

entradas: endereços de dois números racionais n1 e n2 e o endereço do número racional resultante nr.

processo: denominador de nr = denominador de n1 *
denominador de n2;

o numerador de nr = numerador de n1 * denominador de n2 -
numerador de n2 * denominador de n1

saída: 1 se sucesso e 0 se fracasso (se n1 ou n2 ou nr forem
inválidos)

Exemplo de TAD: N_Racionais

- `multiplica_rac`

entradas: endereços de dois números racionais $n1$ e $n2$ e o endereço do número racional resultante nr .

processo: denominador de $nr = \text{denominador de } n1 * \text{denominador de } n2$;
o numerador de $nr = \text{numerador de } n1 * \text{numerador de } n2$

saída: 1 se sucesso e 0 se fracasso (se $n1$ ou $n2$ ou nr forem inválidos)

Exemplo de TAD: N_Racionais

- `divide_rac`

entradas: endereços de dois números racionais $n1$ e $n2$ e o endereço do número racional resultante nr .

processo: denominador de $nr = \text{denominador de } n1 * \text{numerador de } n2$;
o numerador de $nr = \text{numerador de } n1 * \text{denominador de } n2$

saída: 1 se sucesso e 0 se fracasso (se $n1$ ou $n2$ ou nr forem inválidos)

Exemplo de TAD: N_Racionais

- `simplifica_rac`

entradas: endereço de um numero racional

processo: simplifica o numero racional passado como parâmetro

saida: 1 se sucesso e 0 se fracasso (se numero racional for invalido)

Cliente para a TAD N_Racionais

Implemente um programa que leia dois números racionais pela linha de comando e imprima o resultado da soma, da subtração e da divisão destes dois números. Os resultados devem estar na forma simplificada.

Implementação da TAD

N_Racionais

N_Racional.h

N_Racional.c

Referências

- Nina Edelwais e Renata Galante – Estrutura de Dados – Série de Livros Didáticos – Informática – UFRGS.
- Waldemar Celes, Renato Cerqueira e José Lucas Rangel. Introdução a Estrutura de Dados. Ed. Campus (2004).